

# **VE Governance plugin - Audit 4 *Aragon***

# **HALBORN**

# VE Governance plugin - Audit 4 - Aragon

Prepared by:  HALBORN

Last Updated 10/23/2025

Date of Engagement: September 24th, 2025 - October 8th, 2025

## Summary

**100%** ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

| ALL FINDINGS | CRITICAL | HIGH | MEDIUM | LOW | INFORMATIONAL |
|--------------|----------|------|--------|-----|---------------|
| 8            | 0        | 0    | 2      | 3   | 3             |

## TABLE OF CONTENTS

- 1. Summary
- 2. Assessment summary
- 3. Test approach and methodology
- 4. Risk methodology
- 5. Scope
- 6. Assessment summary & findings overview
- 7. Findings & Tech Details
  - 7.1 Timing logic errors in withdrawal and merge operations
  - 7.2 Retroactive fee parameter changes expose users to unexpected charges
  - 7.3 Mincooldown can be zero even when early exit is disabled in fixed fee mode
  - 7.4 Maximum fee can only apply at exact mincooldown due to exit restrictions
  - 7.5 Missing access control in createlockfor() function
  - 7.6 Floating and outdated pragma
  - 7.7 Missing zero address checks
  - 7.8 Missing events

## 1. Summary

Aragon engaged Halborn to conduct a security assessment on their smart contracts beginning on September 24th, 2025 and ending on October 8th, 2025. The scope of this assessment was limited to the smart contracts provided to the Halborn team. Commit hashes and additional details are documented in the Scope section of this report.

The VE Governance Plugin centers around a modular Voting Escrow (ve) system that uses veNFTs (ERC721) to represent locked voting power, rewarding users for longer lock durations. Key contracts include VotingEscrowIncreasing.sol for managing locks and voting power, and GaugeVoter.sol for recording votes across customizable options. Users can lock tokens, receive veNFTs, and gain voting power over time, with features like exit queues, delegation, and vote splitting/merging. The system is highly configurable via custom curves, clocks, and exit logic, enabling flexible governance models tailored to each DAO's needs.

## 2. Assessment Summary

**Halborn** was provided 10 days for the engagement and assigned 1 full-time security engineer to review the security of the smart contracts in scope. The engineer is a blockchain and smart contract security expert with advanced penetration testing and smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the smart contracts.
- Ensure that smart contract functionality operates as intended.

In summary, during the assessment **Halborn** identified several areas for improvement to reduce both the likelihood and impact of potential risks, which were partially addressed by the **Aragon team**. The primary suggestions include:

- Use strict inequality ( $>$ ) in `canMerge` for maturity checks, and replace checkpoint-based withdrawal validation with a dedicated withdrawal mapping.
- Snapshot parameters per ticket which ensures users always exit under the exact terms they agreed to when queuing.
- Add a validation check to prevent setting `_cooldown == 0` when `_allowEarlyExit == false`.
- Replace the misleading conditional check with a stricter and more accurate check.
- Implement role-based access control by adding a `LOCK_CREATOR_ROLE` and using the `auth()` modifier on the `createLockFor()`.

### 3. Test Approach And Methodology

**Halborn** performed a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of this assessment. While manual testing is essential to uncover flaws in logic, process, and implementation, automated testing techniques enhance coverage of smart contracts and can quickly identify issues that do not follow security best practices.

The following phases and associated tools were used throughout the assessment:

- Research into the architecture, purpose, and use of the platform.
- Manual code review and walkthrough of the smart contracts to identify potential logic issues.
- Manual testing of all core functions, including **createCampaign**, claim to validate expected behavior and identify edge-case vulnerabilities.
- Local testing to simulate contract interactions and validate functional and security assumptions.
- Local deployment and testing with Foundry.

## 4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

### 4.1 EXPLOITABILITY

#### ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

#### ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

#### ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

#### METRICS:

| EXPLOITABILITY METRIC ( $M_E$ ) | METRIC VALUE                               | NUMERICAL VALUE   |
|---------------------------------|--------------------------------------------|-------------------|
| Attack Origin (AO)              | Arbitrary (AO:A)<br>Specific (AO:S)        | 1<br>0.2          |
| Attack Cost (AC)                | Low (AC:L)<br>Medium (AC:M)<br>High (AC:H) | 1<br>0.67<br>0.33 |



| EXPLOITABILITY METRIC ( $M_E$ ) | METRIC VALUE                               | NUMERICAL VALUE   |
|---------------------------------|--------------------------------------------|-------------------|
| Attack Complexity (AX)          | Low (AX:L)<br>Medium (AX:M)<br>High (AX:H) | 1<br>0.67<br>0.33 |

Exploitability  $E$  is calculated using the following formula:

$$E = \prod m_e$$

## 4.2 IMPACT

### CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

### INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

### AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

### DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

### YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

### METRICS:

| IMPACT METRIC ( $M_I$ ) | METRIC VALUE                                                            | NUMERICAL VALUE               |
|-------------------------|-------------------------------------------------------------------------|-------------------------------|
| Confidentiality (C)     | None (C:N)<br>Low (C:L)<br>Medium (C:M)<br>High (C:H)<br>Critical (C:C) | 0<br>0.25<br>0.5<br>0.75<br>1 |

| IMPACT METRIC ( $M_I$ ) | METRIC VALUE   | NUMERICAL VALUE |
|-------------------------|----------------|-----------------|
| Integrity (I)           | None (I:N)     | 0               |
|                         | Low (I:L)      | 0.25            |
|                         | Medium (I:M)   | 0.5             |
|                         | High (I:H)     | 0.75            |
|                         | Critical (I:C) | 1               |
| Availability (A)        | None (A:N)     | 0               |
|                         | Low (A:L)      | 0.25            |
|                         | Medium (A:M)   | 0.5             |
|                         | High (A:H)     | 0.75            |
|                         | Critical (A:C) | 1               |
| Deposit (D)             | None (D:N)     | 0               |
|                         | Low (D:L)      | 0.25            |
|                         | Medium (D:M)   | 0.5             |
|                         | High (D:H)     | 0.75            |
|                         | Critical (D:C) | 1               |
| Yield (Y)               | None (Y:N)     | 0               |
|                         | Low (Y:L)      | 0.25            |
|                         | Medium (Y:M)   | 0.5             |
|                         | High (Y:H)     | 0.75            |
|                         | Critical (Y:C) | 1               |

Impact  $I$  is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

### 4.3 SEVERITY COEFFICIENT

#### REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

#### SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

#### METRICS:

| SEVERITY COEFFICIENT ( $C$ ) | COEFFICIENT VALUE | NUMERICAL VALUE |
|------------------------------|-------------------|-----------------|
| Reversibility ( $r$ )        | None (R:N)        | 1               |
|                              | Partial (R:P)     | 0.5             |
|                              | Full (R:F)        | 0.25            |
| Scope ( $s$ )                | Changed (S:C)     | 1.25            |
|                              | Unchanged (S:U)   | 1               |



Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

| SEVERITY      | SCORE VALUE RANGE |
|---------------|-------------------|
| Critical      | 9 - 10            |
| High          | 7 - 8.9           |
| Medium        | 4.5 - 6.9         |
| Low           | 2 - 4.4           |
| Informational | 0 - 1.9           |

5. SCOPE

| REPOSITORY                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>(a) Repository: <a href="#">ve-governance</a></div><div>(b) Assessed Commit ID: <a href="#">f743c12</a></div><div>(c) Items in scope:<ul style="list-style-type: none"><li>• <a href="#">Clock_v1_2_0.sol</a></li><li>• <a href="#">IClock_v1_2_0.sol</a></li><li>• <a href="#">LinearIncreasingCurve.sol</a></li><li>• <a href="#">LinearIncreasingCurveNoSupply.sol</a></li><li>• <a href="#">IEscrowCurveIncreasing_v1_2_0.sol</a></li><li>• <a href="#">EscrowIVotesAdapter.sol</a></li><li>• <a href="#">IEscrowIVotesAdapter.sol</a></li><li>• <a href="#">VotingEscrowIncreasing_v1_2_0.sol</a></li><li>• <a href="#">IVotingEscrowIncreasing_v1_2_0.sol</a></li><li>• <a href="#">GaugesDaoFactory_v1_2_0.sol</a></li><li>• <a href="#">GaugesDaoFactory_v1_3_0.sol</a></li><li>• <a href="#">Lock_v1_2_0.sol</a></li><li>• <a href="#">GaugeVoterSetup_v1_2_0.sol</a></li><li>• <a href="#">GaugeVoterSetup_v1_3_0.sol</a></li><li>• <a href="#">AddressGaugeVoter.sol</a></li><li>• <a href="#">IAddressGaugeVoter.sol</a></li><li>• <a href="#">GaugesDaoFactory_v1_4_0.sol</a></li><li>• <a href="#">GaugeVoterSetup_v1_4_0.sol</a></li><li>• <a href="#">DelegationHelper.sol</a></li><li>• <a href="#">DynamicExitQueue.sol</a></li></ul></div><div>Out-of-Scope: Third party dependencies and economic attacks.</div></div> |

| REMEDIATION COMMIT ID:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><ul style="list-style-type: none"><li>• <a href="https://github.com/aragon/ve-governance/pull/177">https://github.com/aragon/ve-governance/pull/177</a></li><li>• <a href="https://github.com/aragon/ve-governance/pull/180">https://github.com/aragon/ve-governance/pull/180</a></li><li>• <a href="https://github.com/aragon/ve-governance/pull/179">https://github.com/aragon/ve-governance/pull/179</a></li><li>• <a href="https://github.com/aragon/ve-governance/pull/178">https://github.com/aragon/ve-governance/pull/178</a></li></ul></div> <div>Out-of-Scope: New features/implementations after the remediation commit IDs.</div> |

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

**CRITICAL**  
**0**

**HIGH**  
**0**

**MEDIUM**  
**2**

**LOW**  
**3**

**INFORMATIONAL**  
**3**

| SECURITY ANALYSIS                                                          | RISK LEVEL    | REMEDIATION DATE           |
|----------------------------------------------------------------------------|---------------|----------------------------|
| TIMING LOGIC ERRORS IN WITHDRAWAL AND MERGE OPERATIONS                     | MEDIUM        | SOLVED - 10/15/2025        |
| RETROACTIVE FEE PARAMETER CHANGES EXPOSE USERS TO UNEXPECTED CHARGES       | MEDIUM        | SOLVED - 10/15/2025        |
| MINCOOLDOWN CAN BE ZERO EVEN WHEN EARLY EXIT IS DISABLED IN FIXED FEE MODE | LOW           | SOLVED - 10/15/2025        |
| MAXIMUM FEE CAN ONLY APPLY AT EXACT MINCOOLDOWN DUE TO EXIT RESTRICTIONS   | LOW           | RISK ACCEPTED - 10/16/2025 |
| MISSING ACCESS CONTROL IN CREATELOCKFOR() FUNCTION                         | LOW           | SOLVED - 10/15/2025        |
| FLOATING AND OUTDATED PRAGMA                                               | INFORMATIONAL | ACKNOWLEDGED - 10/16/2025  |
| MISSING ZERO ADDRESS CHECKS                                                | INFORMATIONAL | ACKNOWLEDGED - 10/16/2025  |
| MISSING EVENTS                                                             | INFORMATIONAL | ACKNOWLEDGED - 10/16/2025  |

## 7. FINDINGS & TECH DETAILS

### 7.1 TIMING LOGIC ERRORS IN WITHDRAWAL AND MERGE OPERATIONS

// MEDIUM

#### Description

The `VotingEscrowV1_2_0` contract contains timing logic errors that incorrectly restrict legitimate user operations. These issues manifest in two distinct scenarios: (1) blocking valid split-then-withdraw operations due to improper checkpoint timestamp tracking, and (2) preventing the merging of tokens that have exactly reached maturity due to an off-by-one error in the maturity check. Both issues stem from boundary condition errors in timestamp comparisons and prevent users from performing operations that should be immediately available according to the intended business logic.

#### Issue 1: Withdrawal Lock Incorrectly Blocks Split-Then-Withdraw Operations

When the `split()` function was called, it invoked `_checkpoint()`, which updated the token's checkpoint in the curve contract with `writtenTs = block.timestamp`.

```
569 | IEscrowCurve.TokenPoint memory point = IEscrowCurve(curve).tokenPointHistory(_tokenId, 1);
570 | if (block.timestamp == point.writtenTs || block.timestamp == mergeWithdrawalLock[_tokenId]) {
571 |     revert CannotWithdrawInSameBlock();
572 | }
```

Consequently, when `beginWithdrawal()` was called in the same block, the check would incorrectly revert, even though the token was originally created in a previous block and only underwent a split operation.

#### Issue 2: Off-by-One Error Prevents Merging of Mature Tokens

The maturity validation logic in `canMerge()` uses an incorrect comparison operator.

```
445 | if (
446 |     (_toLocked.start != _fromLocked.start) &&
447 |     (toLockedEnd >= block.timestamp || fromLockedEnd >= block.timestamp)
448 | ) {
449 |     return false;
450 | }
```

The `>=` operator prevents merging when either token's end time equals the current timestamp. However, a token that has `endTime == block.timestamp` has reached maturity and should be eligible for merging. This off-by-one error incorrectly treats tokens at the exact moment of maturity as if they are still actively locked, preventing legitimate merge operations.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (5.0)

## Recommendation

Update the maturity check in the `canMerge` function to use a strict inequality ( `>` ) instead of a non-strict one ( `>=` ) when comparing token lock end times against the current block timestamp and replace the checkpoint-based withdrawal validation with a dedicated mapping that explicitly tracks token creation timing rather than modification timing.

## Remediation Comment

**SOLVED:** The **Aragon team** resolved the issue by removing the checkpoint-based validation logic and introducing a dedicated `withdrawalLock` mapping that explicitly tracks only token creation timestamps.

## Remediation Hash

<https://github.com/aragon/ve-governance/pull/177>

## 7.2 RETROACTIVE FEE PARAMETER CHANGES EXPOSE USERS TO UNEXPECTED CHARGES

// MEDIUM

### Description

The `DynamicExitQueue` contract does not snapshot key parameters (such as fees or cooldown periods) at the time users enter the exit queue. As a result, subsequent administrative updates to these parameters can retroactively impact users already in the queue. This may lead to situations where users face higher fees or longer cooldowns than they anticipated based on the conditions at the time of joining the queue.

The exit queue system calculates fees and cooldown eligibility using **current global parameters** rather than the parameters that existed when a user queued their exit. When administrators update fee structures or cooldown periods via `setDynamicExitFeePercent()`, `setTieredExitFeePercent()`, or `setFixedExitFeePercent()`, all existing queued positions are immediately subject to the new parameters without any protection.

The attack scenario is as follows:

- Initial state: Fixed 5% exit fee, 7-day cooldown
- User calls `queueExit()` expecting to pay 5% after 7 days
- User waits 3 days
- Admin calls `setDynamicExitFeePercent()` with parameters:
  - `_minFeePercent = 200` (2%)
  - `_maxFeePercent = 1000` (10%)
  - `_cooldown = 30 days`
  - `_minCooldown = 1 day`
- User attempts to exit at day 3
- **Expected:** 5% fee
- **Actual:** 10% fee
- **Result:** User pays 2x more than expected

### Code Location

The `queueExit()` function only stores minimal ticket data without snapshotting fee parameters:

```
314 function calculateFee(uint256 _tokenId) public view returns (uint256) {
315     TicketV2 memory ticket = _queue[_tokenId];
316     if (ticket.holder == address(0)) return 0;
317
318     uint256 underlyingBalance = IVotingEscrow(escrow).locked(_tokenId).amount;
319     if (underlyingBalance == 0) revert NoLockBalance();
320
321     uint256 timeElapsed = block.timestamp - ticket.queuedAt;
322     uint256 scaledFeePercent = _getScaledTimeBasedFee(timeElapsed);
323     return (underlyingBalance * scaledFeePercent) / INTERNAL_PRECISION;
324 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

### Recommendation

Snapshot parameters per ticket which ensures users always exit under the exact terms they agreed to when queuing, preventing retroactive fee increases or cooldown extensions.

### Remediation Comment

**SOLVED:** The Aragon team resolved the issue by implementing parameter snapshotting in the **TicketV2** struct.

### Remediation Hash

<https://github.com/aragon/ve-governance/pull/180>



## 7.3 MINCOOLDOWN CAN BE ZERO EVEN WHEN EARLY EXIT IS DISABLED IN FIXED FEE MODE

// LOW

### Description

In the `DynamicExitQueue::_setFixedExitFeePercent` function, the `minCooldown` is set to `_cooldown` when early exits are disallowed (`_allowEarlyExit == false`). However, there is no check to ensure `_cooldown` is greater than zero in that case. If `_cooldown == 0` and `_allowEarlyExit == false`, then `minCooldown = 0`, which effectively **allows immediate exit**, directly contradicting the intention of disabling early exits.

### Code Location

In `DynamicExitQueue::_setFixedExitFeePercent`, `minCooldown` is set to `_cooldown` without checking if it's non-zero when early exit is disabled:

```
207 function _setFixedExitFeePercent(  
208     uint256 _feePercent,  
209     uint48 _cooldown,  
210     bool _allowEarlyExit  
211 ) internal {  
212     feePercent = _feePercent;  
213     minFeePercent = _feePercent;  
214     cooldown = _cooldown;  
215     _slope = 0; // No decay in fixed system  
216  
217     // immediate or none  
218     if (_allowEarlyExit) minCooldown = 0;  
219     else minCooldown = _cooldown;  
220  
221     emit ExitFeePercentAdjusted(_feePercent, _feePercent, minCooldown, ExitFeeType.Fixed);  
222 }
```

### BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

### Recommendation

Add a validation check to prevent setting `_cooldown == 0` when `_allowEarlyExit == false`.

### Remediation Comment

**SOLVED:** The Aragon team resolved the issue by removing optional early exit control and applying single fee rate to all exits after the minimum cooldown period.

### Remediation Hash

<https://github.com/aragon/ve-governance/pull/179>

## 7.4 MAXIMUM FEE CAN ONLY APPLY AT EXACT MINCOOLDOWN DUE TO EXIT RESTRICTIONS

// LOW

### Description

In `DynamicExitQueue::_getScaledTimeBasedFee`, users are only allowed to exit if `timeElapsed >= minCooldown`, enforced by the `canExit()` function. However, the internal fee calculation logic (`_getScaledTimeBasedFee`) checks:

```
339 | if (timeElapsed <= minCooldown) return scaledMaxFee;
```

Since `timeElapsed < minCooldown` is **never reachable** in `exit()`, the above condition is effectively only ever true when `timeElapsed == minCooldown`. This means the **maximum fee** (`scaledMaxFee`) is **only applied at a single second**, and decay starts immediately after. The use of `<=` gives the false impression that a wider time range qualifies for maximum fee, which is misleading and unnecessary.

### BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:L (2.5)

### Recommendation

Replace the misleading conditional check with a stricter and more accurate check:

```
if (timeElapsed == minCooldown) return scaledMaxFee;
```

### Remediation Comment

**RISK ACCEPTED:** The Aragon team made a business decision to accept the risk of this finding and not alter the contracts.

## 7.5 MISSING ACCESS CONTROL IN CREATELOCKFOR() FUNCTION

// LOW

### Description

The `VotingEscrowV1_2_0::createLockFor()` function claims to be "Restricted by default" according to its documentation, but the implementation lacks any access control mechanism. This allows any address to create locks on behalf of other users without their consent, enabling griefing attacks and unwanted NFT spam.

The `createLockFor` function comment states it should be restricted, but the actual implementation is publicly callable without any authorization checks:

```
330 | /// @notice Creates a lock on behalf of someone else. Restricted by default.
331 | function createLockFor(
332 |     uint256 _value,
333 |     address _to
334 | ) external nonReentrant whenNotPaused returns (uint256) {
335 |     return _createLockFor(_value, _to);
336 | }
```

The function only has `nonReentrant` and `whenNotPaused` modifiers, but no access control modifier such as `auth(ROLE)` or whitelist checks that would restrict who can call it.

### BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

### Recommendation

Implement role-based access control by adding a `LOCK_CREATOR_ROLE` and using the `auth()` modifier on the `createLockFor()` function to restrict it to authorized addresses only. Alternatively, if unrestricted access is intentional, update the documentation to remove the "Restricted by default" claim and clearly state the function is publicly callable.

### Remediation Comment

**SOLVED:** The Aragon team resolved the issue by updating the comment to reflect the intended behavior.

### Remediation Hash

<https://github.com/aragon/ve-governance/pull/178>

## 7.6 FLOATING AND OUTDATED PRAGMA

// INFORMATIONAL

### Description

Locking the `pragma` helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities. The contract allowed floating or unlocked `pragma` to be used, i.e., `>= 0.8.17`. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified.

### Code Location

Floating and outdated `pragma` version in all contracts.

```
1 | /// SPDX-License-Identifier: MIT
2 | pragma solidity ^0.8.17;
```

If the smart contract gets compiled and deployed with an older or too recent version of the solidity compiler, there's a chance that it may get compromised due to the bugs present in the older versions or unidentified exploits in the new versions. Incompatibility issues may also arise if the contract code does not support features in other compiler versions, therefore, breaking the logic.

### BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

### Recommendation

Keep the compiler versions consistent in all the smart contract files. Do not allow floating `pragmas` anywhere. It is suggested to use the 0.8.28 `pragma` version.

### Remediation Comment

**ACKNOWLEDGED:** The Aragon team acknowledged this finding, also mentioning that:

| Need floating `pragma` as `ve` is used as dependency in lots of our builds.

## 7.7 MISSING ZERO ADDRESS CHECKS

// INFORMATIONAL

### Description

The contract `VotingEscrowV1_2_0` and `AddressGaugeVoter` does not include explicit checks for zero addresses when setting critical contract dependencies such as the `ivotesAdapter` and the `token` or `clock` during initialization. This opens up the contract to potential misconfigurations, leading to unexpected behavior or denial of core functionalities.

### Code Location

There is no address zero check:

```
170 | function setIVotesAdapter(  
171 |     address _ivotesAdapter  
172 | ) external auth(ESCROW_ADMIN_ROLE) contractAlreadySet(ivotesAdapter) {  
173 |     ivotesAdapter = _ivotesAdapter;  
174 | }
```

### BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (1.7)

### Recommendation

Add zero address validations.

### Remediation Comment

**ACKNOWLEDGED:** The Aragon team acknowledged this finding, also mentioning that:

These are set by the DAO and factories so the validation is the governance process.

## 7.8 MISSING EVENTS

// INFORMATIONAL

### Description

The `AddressGaugeVoter` and `DynamicExitQueuecontract` performs critical state updates in the following functions:

- `setIVotesAdapter(address _ivotesAdapter)`
- `setEnableUpdateVotingPowerHook(bool _enableUpdateVotingPowerHook)`
- `withdraw`

However, **no events are emitted** when these updates occur. This breaks **best practices** for smart contract transparency and observability. Off-chain systems (such as indexers, dashboards, or governance tools) rely on events to detect and react to important state changes.

### BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (1.7)

### Recommendation

Add events for each function wherever state transition happens.

### Remediation Comment

**ACKNOWLEDGED:** The **Aragon team** acknowledged this finding, also mentioning that:

ivotesadapter can't be set more than once, setEnableHook will typically never be changed, withdraw emits ERC20::Transfer event.

---

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.